

MÓDULO 4 – HERANÇA E REUSO

Herança, Sobrescrita, **super** & Polimorfismo

Como fazer classes nascerem de outras classes, reaproveitar código com inteligência e escrever programas que se adaptam sozinhos ao tipo de objeto.

`def __init__``class Filho(Pai)``super()`

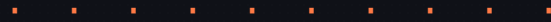
O QUE VAMOS PERCORRER HOJE

Roteiro da aula

BLOCO	TEMA
01	O problema: código repetido
02	Herança: o conceito
03	Classe base × classe derivada
04	Quando (e quando não) usar
05	Sobrescrita de métodos

BLOCO	TEMA
06	O <code>super()</code> — estender, não substituir
07	Polimorfismo
08	Ligação dinâmica (a "mágica")
09	Exercícios (8 no total)
10	Recapitulação

 **4 horas de aula.** Sugestão de corte: **Parte A** (Herança + `super`) e **Parte B** (Sobrescrita + Polimorfismo). Exercícios distribuídos ao longo e concentrados no fim.



01

Herança

Reaproveitar código sem copiar e colar. A primeira grande ferramenta de reuso da orientação a objetos.

ANTES DA SOLUÇÃO, O PROBLEMA

"Copiar e colar" não escala

Imagine um sistema com **Cachorro** e **Gato**. Os dois têm nome, idade e comem. Sem herança:

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
    def comer(self):
        print(f"{self.nome} está comendo")
    def latir(self):
        print("Au au!")
```

```
class Gato:
    def __init__(self, nome, idade):
        self.nome = nome      # repetido!
        self.idade = idade    # repetido!
    def comer(self):          # repetido!
        print(f"{self.nome} está comendo")
    def miar(self):
        print("Miau!")
```

● Se amanhã eu adicionar peso ou corrigir um bug em `comer()`, preciso mexer em **todas** as classes. Quanto mais animais, pior. Tem que existir um jeito melhor.

A IDEIA, ANTES DO CÓDIGO

Você já herda coisas

Pense em **herança genética**. Você herdou características dos seus pais: cor dos olhos, altura, tipo sanguíneo. Você **não precisou inventar** essas coisas — elas já vieram prontas.

Mas você também tem características **suas**, próprias, que seus pais não têm. E às vezes você faz algo **do seu jeito**, diferente de como eles fariam.



Em POO é igual: uma classe pode **herdar** tudo de outra, **adicionar** o que é só dela e **fazer diferente** quando quiser.

Outros exemplos do dia a dia

- ◆ **Veículo** → Carro, Moto, Caminhão
- ◆ **Funcionário** → Gerente, Vendedor, Estagiário
- ◆ **Conta bancária** → Conta Corrente, Poupança
- ◆ **Forma geométrica** → Círculo, Quadrado

O termo-chave é a relação "**é um tipo de**": um Carro **é um** Veículo. Um Gerente **é um** Funcionário.

DEFINIÇÃO

Herança é o mecanismo pelo qual uma classe (a **filha**) recebe automaticamente os atributos e métodos de outra classe (a **mãe**).

Classe base / mãe / superclasse

A classe genérica, com o que é **comum** a todos. Ex.:
Animal

Classe derivada / filha / subclasse

A classe especializada, que herda e **acrescenta** o específico. Ex.: Cachorro

Sinônimos que você vai encontrar por aí: **base/derivada**, **pai/filho**, **super/sub**. São a mesma ideia.

COMO SE ESCRIVE EM PYTHON

A mesma coisa, agora com herança

```
# Classe BASE (mãe)
class Animal:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
    def comer(self):
        print(f"{self.nome} está comendo")

# Classe DERIVADA (filha)
class Cachorro(Animal): # ← herda!
    def latir(self):
        print("Au au!")

class Gato(Animal): # ← herda!
    def miar(self):
        print("Miau!")
```

```
>>> rex = Cachorro("Rex", 3)
>>> rex.nome
'Rex' # herdado de Animal
>>> rex.comer()
Rex está comendo # herdado!
>>> rex.latir()
Au au! # próprio do Cachorro
```

✓ A palavra mágica é o **parênteses no nome da classe**:

`class Cachorro(Animal):`

Lê-se: "**Cachorro é um Animal**". Zero repetição de nome, idade ou comer.

O QUE A FILHA REALMENTE GANHA

A filha herda tudo o que é público

Atributos

Os dados definidos na mãe (ex.: nome, idade) passam a existir na filha.

Métodos

Os comportamentos da mãe (ex.: comer()) podem ser chamados na filha.

+ o que é só dela

A filha pode declarar atributos e métodos novos, exclusivos dela.

 **Regra de ouro da busca:** quando você chama `objeto.algo()`, o Python procura primeiro na **classe do objeto**. Se não achar, sobe para a **mãe**. Se não achar, sobe de novo, até chegar em `object`. Isso se chama **cadeia / árvore de herança**.

Cachorro



Animal



object

(toda classe Python herda de object no fim)

A PERGUNTA QUE DECIDE TUDO

Quando usar herança?

✓ Use quando faz sentido dizer "É UM"

- ◆ Um Cachorro é um Animal ✓
- ◆ Um Gerente é um Funcionário ✓
- ◆ Um Quadrado é uma Forma ✓

Se a frase soa natural em português, a herança provavelmente é adequada.

✗ Desconfie quando for "TEM UM"

- ◆ Um Carro ~~é um~~ tem um Motor ✗
- ◆ Uma Casa ~~é uma~~ tem uma Porta ✗
- ◆ Um Pedido ~~é um~~ tem um Cliente ✗

"Tem um" não é herança. (Esse caso resolve-se de outra forma — assunto de aulas futuras.)

🎯 **Teste do "É UM":** antes de escrever `class B(A)`, diga em voz alta "**B é um A**". Se não fizer sentido, não use herança.

CASO REAL: SISTEMA DE RH

Funcionários de uma empresa

```
class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario

    def apresentar(self):
        print(f"Sou {self.nome}, ganho R${self.salario}")

class Vendedor(Funcionario):
    def registrar_venda(self, valor):
        comissao = valor * 0.05
        print(f"Comissão: R${comissao}")

class Gerente(Funcionario):
    def aprovar_ferias(self, quem):
        print(f"Férias de {quem} aprovadas")
```

```
>>> v = Vendedor("Ana", 3000)
>>> v.apresentar()          # herdado
Sou Ana, ganho R$3000
>>> v.registrar_venda(2000) # próprio
Comissão: R$100.0

>>> g = Gerente("Beto", 8000)
>>> g.apresentar()          # herdado
Sou Beto, ganho R$8000
>>> g.aprovar_ferias("Ana")
Férias de Ana aprovadas
```

O código de nome, salario e apresentar() foi escrito **uma vez** e reaproveitado por **todos**.

PRATIQUE AGORA – HERANÇA BÁSICA

Exercícios 1 e 2

1

Veículos

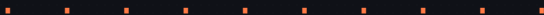
Crie uma classe `Veiculo` com atributos `marca` e `ano`, e um método `ligar()` que imprime "Veículo ligado". Depois crie `Carro` e `Moto` que herdam de `Veiculo`. Adicione um método `abrir_porta_malas()` só ao `Carro`.

2

Conta bancária

Crie `Conta` com `titular` e `saldo`, e métodos `depositar(v)` e `ver_saldo()`. Crie `ContaPoupanca(Conta)` com um método novo `render_juros(taxa)` que aumenta o saldo. Teste criando uma poupança e rendendo juros.

🕒 ~15 min. Dica: você **não** precisa reescrever `__init__` na filha se ela não tiver atributos novos — ela usa o da mãe automaticamente.



02

A função `super()`

"E se a filha precisar de atributos novos, mas **também** quiser aproveitar a inicialização da mãe?"

O PROBLEMA QUE O SUPER RESOLVE

A filha precisa de mais dados

Um **Cachorro** tem nome e idade (como todo **Animal**), mas também tem uma **raca**. Como inicializar os três?

TENTATIVA INGÊNUA (REPETE TUDO) ●

```
class Cachorro(Animal):  
    def __init__(self, nome, idade, raca):  
        self.nome = nome    # repetindo  
        self.idade = idade  # a mãe!  
        self.raca = raca
```

De novo o copia-e-cola. Se a mãe mudar, isto quebra.

COM SUPER() ✓

```
class Cachorro(Animal):  
    def __init__(self, nome, idade, raca):  
        super().__init__(nome, idade)  
        # ↑ a MÃE cuida de nome e idade  
        self.raca = raca    # só o que é novo
```

`super()` é uma referência à **classe mãe**. Chamar `super().__init__(...)` executa o construtor dela.

ENTENDENDO O SUPER COM UMA IMAGEM

"Faz a parte que é tua, mãe"

Imagine montar um **hambúrguer artesanal**. O hambúrguer comum já tem um preparo: pão, carne, queijo. O **cheeseburger bacon** não joga tudo fora — ele **pede o hambúrguer comum primeiro** e só **adiciona o bacon** em cima.

 `super().__init__()` = "monta o hambúrguer base".
O resto do `__init__` da filha = "agora adiciono o bacon".

A filha **estende** o trabalho da mãe em vez de refazê-lo do zero.

O fluxo do construtor

```
Cachorro("Rex", 3, "Vira-lata")
```

↓ **chama**

```
super().__init__("Rex", 3)
```

↓ **executa Animal**

```
self.nome / self.idade definidos
```

↓ **volta**

```
self.raca = "Vira-lata"
```

SUPER() NÃO SERVE SÓ PARA O __INIT__

Estendendo qualquer método

Você pode chamar `super().qualquer_metodo()` para aproveitar o comportamento da mãe e **acrescentar** algo.

```
class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario
    def apresentar(self):
        print(f"Sou {self.nome}")

class Gerente(Funcionario):
    def __init__(self, nome, salario, equipe):
        super().__init__(nome, salario)
        self.equipe = equipe
    def apresentar(self):
        super().apresentar() # faz o da mãe
        print(f"Lidero {self.equipe} pessoas")
```

```
>>> g = Gerente("Beto", 8000, 5)
>>> g.apresentar()
Sou Beto           # veio da mãe
Lidero 5 pessoas   # acréscimo do filho
```

🔑 Padrão mental:

"Faça tudo o que a mãe faria" → `super().metodo()`

"...e mais isto aqui" → suas linhas extras.

PRATIQUE AGORA – SUPER()

Exercício 3

3

Pessoa → Estudante

Crie uma classe Pessoa com nome e idade, e um método `descrever()` que imprime nome e idade.

Crie `Estudante(Pessoa)` que **também** recebe curso. Use `super().__init__()` para reaproveitar nome e idade.

Sobrescreva `descrever()` no `Estudante` para chamar `super().descrever()` e, em seguida, imprimir o curso.

RESULTADO ESPERADO

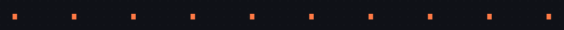
```
e = Estudante("Lia", 20, "ADS")
e.descrever()
```

Saída:

Nome: Lia, Idade: 20

Curso: ADS

🕒 ~10 min. Este exercício já mistura `super()` com `sobrescrita` — exatamente o próximo tema. 😊



03

Sobrescrita de métodos

Quando a filha precisa fazer **do seu próprio jeito** algo que a mãe já fazia.

OVERRIDE / REDEFINIÇÃO

Herdar não é obrigatório obedecer

Até agora a filha **aceitava** os métodos da mãe como vinham. Mas e se o comportamento precisar ser **diferente**?

Todo animal "faz som". Mas o cachorro faz "**Au au**" e o gato faz "**Miau**". O método tem o **mesmo nome**, mas o **conteúdo muda**.

Sobrescrita (override): a filha define um método com o **mesmo nome** de um da mãe. A versão da filha **substitui** a da mãe — só para objetos daquela filha.

```
class Animal:
    def fazer_som(self):
        print("Som genérico...")

class Cachorro(Animal):
    def fazer_som(self):    # mesmo nome!
        print("Au au!")    # conteúdo novo

class Gato(Animal):
    def fazer_som(self):
        print("Miau!")
```

A REGRA DE BUSCA DECIDE QUEM RESPONDE

Cada objeto responde do seu jeito

```
>>> a = Animal()
>>> a.fazer_som()
Som genérico...

>>> rex = Cachorro()
>>> rex.fazer_som()
Au au!                # usou a versão do Cachorro

>>> mimi = Gato()
>>> mimi.fazer_som()
Miau!                 # usou a versão do Gato
```

Por que funciona?

Lembra da **cadeia de busca**? O Python procura fazer_som primeiro na **classe do objeto**.

- ◆ Em rex, acha em **Cachorro** e para ali → "Au au"
- ◆ Em a, não há override → sobe até **Animal** → "Som genérico"

A versão mais "próxima" do objeto sempre ganha. É só a regra de busca que você já conhece, em ação.

CONFUSÃO CLÁSSICA DE PROVA

Sobrescrita ≠ Sobrecarga

Sobrescrita (Override) é o tema de hoje

Mesma assinatura, **classes diferentes** (mãe e filha). A filha redefine o método da mãe.

```
class Gato(Animal):  
    def fazer_som(self): # override  
        print("Miau")
```

Sobrecarga (Overload) — outra coisa

Mesmo nome, **parâmetros diferentes**, na mesma classe. Comum em Java/C++.

Em **Python não existe sobrecarga clássica**: se você definir o mesmo método duas vezes, a segunda simplesmente substitui a primeira. Python usa valores-padrão e *args no lugar.

 Memorize: **sobrescrita** = reescrever o que a mãe fez (vertical, entre classes). **sobrecarga** = várias versões pelos parâmetros (horizontal, mesma classe).

PRATIQUE AGORA – SOBRESCRITA

Exercícios 4 e 5

4

Formas geométricas

Crie Forma com um método `area()` que imprime "Área não definida". Crie `Quadrado(lado)` e `Circulo(raio)` que **sobrescrevem** `area()` com o cálculo correto. Teste os três.

5

Conta com taxa

A partir da Conta do Ex.2, crie `ContaCorrente` que **sobrescreve** `depositar(v)` para cobrar R\$0,50 de taxa por depósito. Use `super().depositar()` e depois desconte a taxa.

🕒 ~20 min. Para o círculo, use `import math` e `math.pi`. O Ex.5 combina sobrescrita e `super` — exatamente o padrão profissional.

04

Polimorfismo

A recompensa de tudo o que vimos: escrever um código que funciona com **qualquer** tipo de objeto, sem saber qual é.

A PALAVRA, DESMONTADA

Poli + morfismo = muitas formas

"**Poli**" = muitas. "**Morfo**" = forma. Polimorfismo é a capacidade de uma **mesma chamada** (ex.: `fazer_som()`) assumir **muitas formas** dependendo do objeto.

Você já viu o ingrediente principal: a **sobrescrita**. Polimorfismo é **usar isso a seu favor** em código que trata vários tipos de uma vez.

🧠 Analogia: o botão "**play**" do controle. O mesmo botão toca um filme, uma música ou um jogo — **cada aparelho responde do seu jeito**. Você só aperta "play".

O salto mental

Sem polimorfismo você escreveria:

```
if tipo == "cachorro":  
    latir()  
elif tipo == "gato":  
    miar()  
elif tipo == "vaca":  
    mugir() # ...e nunca acaba
```

Com polimorfismo, some todo o if/elif:

```
animal.fazer_som() # resolve sozinho
```

UM LOOP QUE TRATA TODOS OS TIPOS

Um código, vários comportamentos

```
class Animal:
    def fazer_som(self): print("...")

class Cachorro(Animal):
    def fazer_som(self): print("Au au!")

class Gato(Animal):
    def fazer_som(self): print("Miau!")

class Vaca(Animal):
    def fazer_som(self): print("Muu!")

# a mágica: uma lista de tipos diferentes
animais = [Cachorro(), Gato(), Vaca()]

for bicho in animais:
    bicho.fazer_som()    # mesma linha p/ todos
```

```
# Saída:
Au au!
Miau!
Muu!
```

🔥 Repare: o for **não sabe nem se importa** com o tipo de cada bicho. Ele só confia que todo Animal **sabe** fazer_som(). Cada objeto responde com sua própria versão.

Amanhã você cria Pato. Adiciona à lista. O loop continua igual. **Código aberto a extensão.**

TIRANDO A "MÁGICA" DA JOGADA

Ligação dinâmica (dynamic dispatch)

A pergunta certa é: **como o Python sabe qual fazer_som chamar?**

A decisão é tomada **na hora da execução**, olhando o **tipo real** do objeto — não o nome da variável, não o tipo "declarado". Isso se chama **ligação dinâmica**.

📌 É o tipo **real** do objeto que manda. bicho pode "parecer" um Animal genérico, mas se por dentro ele é um Gato, é o fazer_som do Gato que roda.

O que acontece em bicho.fazer_som()

1. Qual é o tipo REAL de bicho agora?



2. É um Gato.



3. Gato tem fazer_som? Sim.



4. Executa a versão do Gato.

Esse processo se repete a cada iteração, para cada objeto.

POLIMORFISMO NEM SEMPRE PRECISA DE HERANÇA

Duck typing: o jeito pythônico

"Se anda como um pato e faz quack como um pato, então é um pato."

Em Python, o que importa **não** é a classe do objeto, e sim **se ele tem o método** que você quer chamar.

Os objetos abaixo **não compartilham mãe nenhuma**, mas o loop funciona igual — porque todos têm `fazer_som()`.

```
class Pato:
    def fazer_som(self): print("Quack!")

class Carro:    # nem é animal!
    def fazer_som(self): print("Bibi!")

for x in [Pato(), Carro()]:
    x.fazer_som()

# Quack!
# Bibi!
```

Herança **organiza** e **garante** o contrato; duck typing mostra que, no fundo, polimorfismo é sobre **capacidade**, não sobre linhagem. Em outras linguagens (Java, C#) a herança costuma ser obrigatória para isso.

CASO REAL: RELATÓRIO DE PAGAMENTOS

Por que isso importa no trabalho

```
class Pagamento:
    def processar(self, valor):
        print("Pagamento genérico")

class Cartao(Pagamento):
    def processar(self, valor):
        print(f"Cobrando R${valor} no cartão")

class Pix(Pagamento):
    def processar(self, valor):
        print(f"Pix instantâneo de R${valor}")

class Boleto(Pagamento):
    def processar(self, valor):
        print(f"Boleto gerado: R${valor}")
```

```
def finalizar_compra(forma, valor):
    # não sabe NEM precisa saber a forma
    forma.processar(valor)

finalizar_compra(Pix(), 150)
finalizar_compra(Cartao(), 150)
finalizar_compra(Boleto(), 150)

# Pix instantâneo de R$150
# Cobrando R$150 no cartão
# Boleto gerado: R$150
```

 A função `finalizar_compra` foi escrita **uma vez e nunca mais muda**, mesmo que a empresa adicione 10 formas de pagamento novas. Isso é manutenção barata.

PRATIQUE AGORA – POLIMORFISMO

Exercícios 6, 7 e 8

6

Zoológico

Reúna Cachorro, Gato e Vaca (todos filhos de Animal, cada um sobrescrevendo fazer_som()) numa lista e percorra com um for chamando o som de cada um.

7

Formas + área total

Use as formas do Ex.4. Faça area() **retornar** o valor (em vez de imprimir). Crie uma lista de formas e uma função que **some** a área de todas, sem usar if por tipo.

8

Desafio 🏆

Crie Funcionario com calcular_pagamento(). Crie Mensalista (salário fixo) e Horista (horas × valor_hora) sobrescrevendo o cálculo. Some a folha de pagamento de uma lista mista.

🕒 ~30 min. O Ex.7 e o Ex.8 são o auge da aula: juntam **herança**, **super**, **sobrescrita** e **polimorfismo** num cenário realista. Quem fizer o 8 entendeu o módulo. ✅

O que levamos hoje

Herança

Uma classe nasce de outra e reaproveita seus atributos e métodos. `class Filho(Pai)`. Teste do "é um".

`super()`

Acessa a mãe para **estender** em vez de refazer. Essencial no `__init__` de filhas com atributos novos.

Sobrescrita

A filha redefine um método da mãe com o mesmo nome. A versão mais próxima do objeto vence.

Polimorfismo

Mesma chamada, formas diferentes. Código que trata muitos tipos sem `if`, graças à ligação dinâmica.

 **O fio que costura tudo:** a **cadeia de busca** de métodos. Herança a constrói, sobrescrita a aproveita, `super` a navega para cima, e o polimorfismo é o resultado visível dela em tempo de execução.

MÓDULO 4 — CONCLUÍDO

"É um" herda.
"Faz diferente" sobrescreve.
"Muitas formas" é polimorfismo.

Próximo passo natural: **classes abstratas** (obrigar as filhas a implementar certos métodos) — a ponte perfeita para reforçar tudo isso.

8 exercícios

4 conceitos

0 copia-e-cola